AttentionLens

Interactive Explainability & Automated Bias-Screening for Vision Models



Project Review 1 · B.Tech CSE (AI & ML), VIT

Jaswanth Krishna Padarthi

Agenda

01	**Problem & Motivation**	Why vision models need a debugger, not just explanations
02	**Design & Architecture**	Unified dual-view LayerMap schema (CNN ⇄ ViT)
03	**Dataset**	ImageNet demo sample — provenance & representation
04	**Preprocessing Results**	Resize → normalize → tensor, with measured stats
05	**Feature Engineering**	Native activations + Grad-CAM, per-block, normalized
06	**EDA Visualizations**	Montage, distributions, concentration, correlations
07	**Demo — M1 Gate**	Live ResNet-50 run on a real image (Samoyed)
08	**Results & Next Steps**	Findings, honest failures, M2 ViT roadmap

Problem & Motivation

Explanations exist — but they don't talk to each other

The field is fragmented

- Attention maps (ViT) answer 'what does the model look at' — but they are class-agnostic.
- Attribution maps (Grad-CAM) answer 'what drove this decision' — class-specific.
- No shared schema: comparing two architectures or two views today means writing bespoke glue.

Bias goes unnoticed

- Spurious correlations (backgrounds, textures, artifacts) silently inflate accuracy.
- Finding them today is manual, ad-hoc, and rarely automated end-to-end.
- Engineers need a verdict — not another heatmap to eyeball.

One framework, two jobs

① A unified CNN ⇄ ViT abstraction — every layer yields a standard LayerMap (native attention + class-conditioned relevance). ② An automated bias-screening layer — attention-quality metrics and intervention experiments that produce a verdict, in the tradition of SHAP / Captum as a deployable tool.

Design — the Dual-View LayerMap

Every layer, two views, one schema



Native view — what the model attends to (mechanism)

- Mean over channels of each block's last conv activation
- Upsampled to input resolution (bilinear)
- Normalized to [0, 1]
- Class-agnostic: shows where the model looks

Class-conditioned view — why the model decided

- Grad-CAM on the same conv, for the predicted class
- ReLU → upsample → normalize to [0, 1]
- Evidence for this specific decision
- Same LayerMap contract — comparable by construction

Dataset

ImageNet provenance, real pipeline ingestion



sample.jpg — Samoyed (ImageNet)

Source	ImageNet (demo sample)
True label	Samoyed
Original size	1546 × 1213 px
Pipeline input	224 × 224 RGB
Extraction	8 LayerMaps (4 blocks × 2 views)
Heavy runs	CPU on Arjun (M1); GPU/Jenga planned (M2+)

Review-1 scope: a single curated sample exercises the full pipeline end-to-end. Screening-scale corpora (planted-bias CUB, ObjectNet) are scheduled in the validation milestone.

Preprocessing Results

Resize → /255 → ImageNet normalize → tensor (measured on Arjun)

load RGB → resize 224×224 → ÷255 → [0,1] → −mean ÷std → [1,3,224,224] fp32

Raw pixel stats (0–1, before normalize)

R	G	B
0.388 ± 0.320	0.431 ± 0.295	0.3

Normalized tensor stats (after −mean ÷std)

R	G	B
−0.421 ± 1.398	−0.110 ± 1.316	−0.170 ± 1.498

- Design point: preprocessing is a lossless, reversible step — tensor_to_image() inverts it exactly for visualization overlay.
- Facts: input [1,3,224,224], dtype float32, on-device; stats above measured on Arjun, python 3.12, torch 2.13 CPU.
- Tests: test_preprocess.py covers round-trip equivalence (hermetic — no downloads).

Feature Engineering I — the LayerMap contract

One dataclass rules all extractors, metrics, screening, UI

Field	Type	Meaning
`layer_name`	str	"layer4.2.conv3" / "block11" / "attn_head_3"
`arch`	str	"cnn" | "vit"
`view`	str	"native" | "class_conditioned"
`model_class`	str	class used for conditioning (argmax for native)
`map`	np.ndarray	H×W float32, normalized [0,1], input resolution
`head_or_channel`	int | None	per-head / per-channel breakdown (future)
`metadata`	dict	method, reduction, entropy, locality — extensible

ModelInspection bundles one (model, image) pair: model_id, image_id, true/pred label, LayerMaps, intervention_results. Verified by test_schema.py (7 tests green on Arjun).

Feature Engineering II — extraction methods

Probing the last conv of each ResNet bottleneck block

RESNET_BLOCK_LAYERS

"layer1.2.conv3" · "layer2.3.conv3" · "layer3.5.conv3" · "layer4.2.conv3"

layer1.2.conv3

256 ch

early texture/edges

layer2.3.conv3

512 ch

patterns/parts

layer3.5.conv3

1024 ch

object parts

layer4.2.conv3

2048 ch

object-level semantics

Native (mechanism)

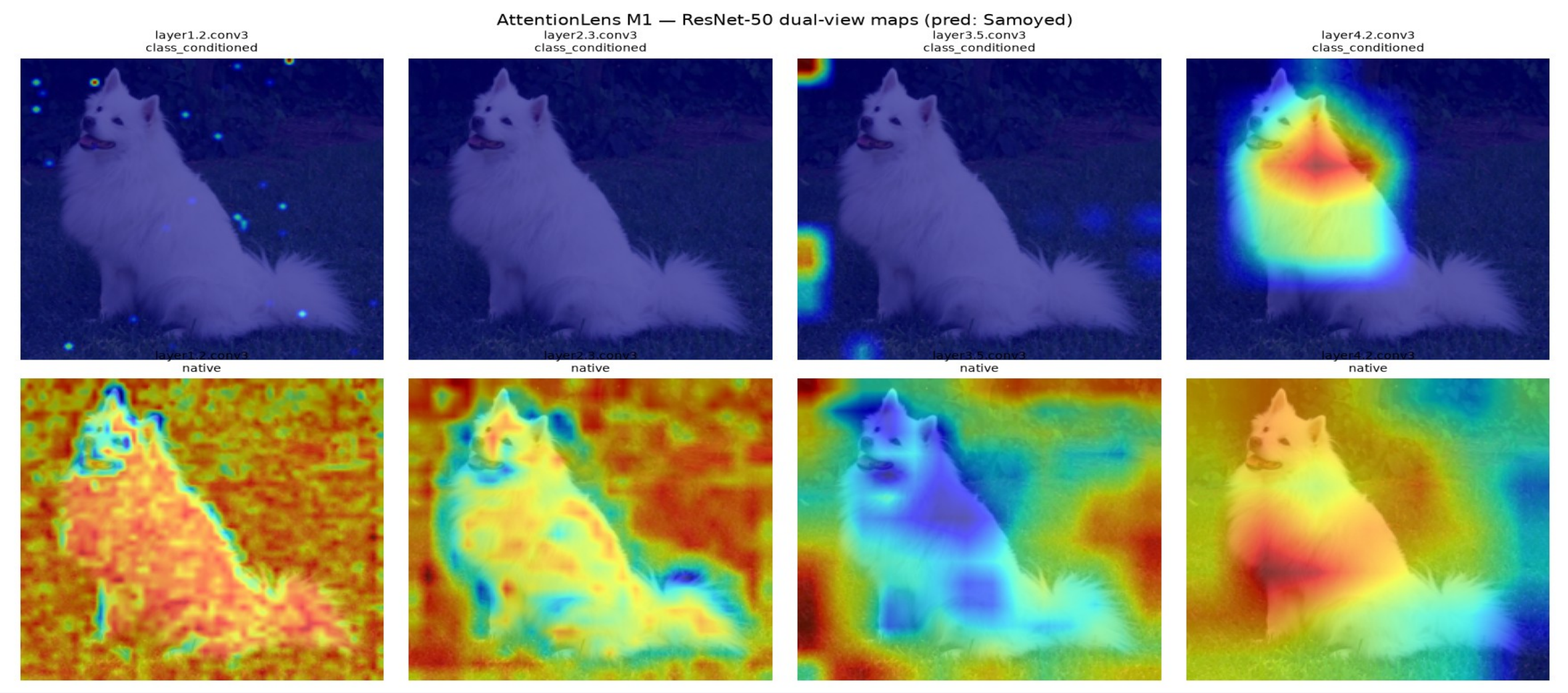
Forward hook → activations[B,C,H,W] → mean(dim=1) → bilinear upsample → min-max normalize. Cost: one forward pass.

Class-conditioned (evidence)

Captum LayerGradCam on the same conv, target = predicted class → ReLU → upsample → normalize. Cost: one attribution per layer.

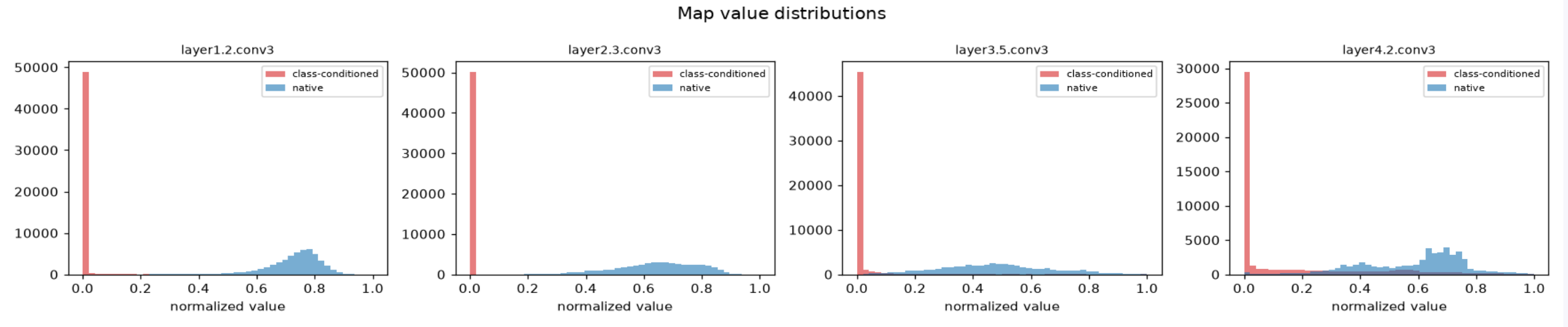
EDA — dual-view montage (live output)

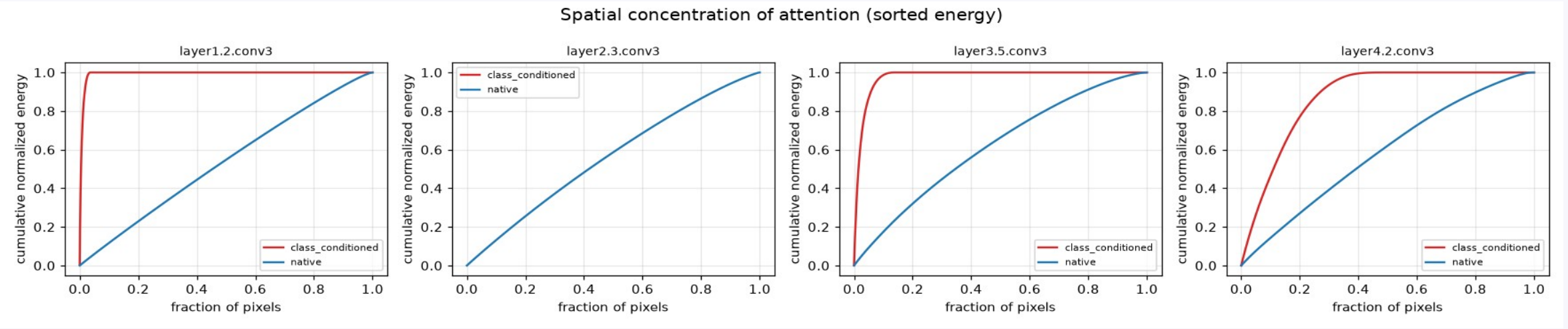
ResNet-50 on sample.jpg — predicted: Samoyed ✓



EDA — value distributions & spatial concentration

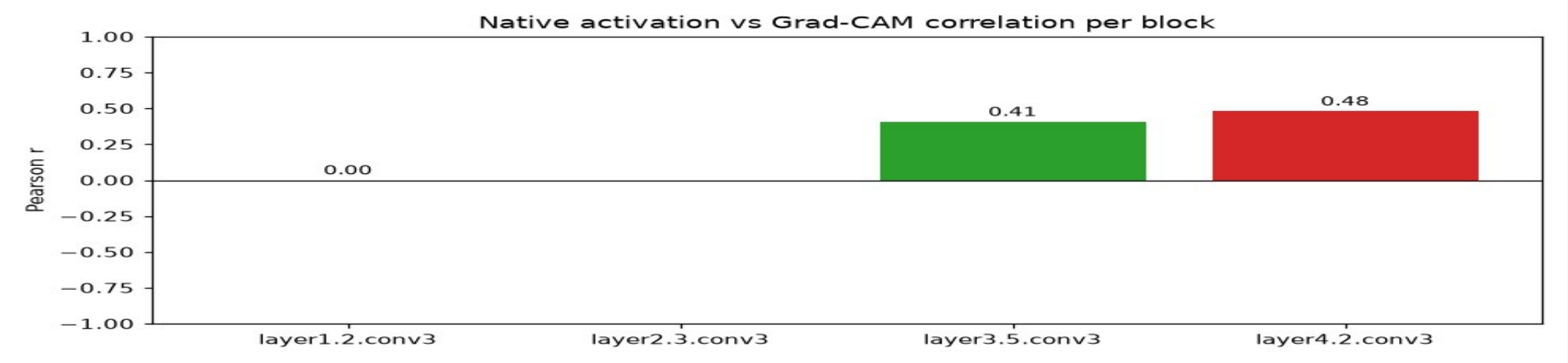
Why the two views can't be swapped





EDA — cross-view correlation (the key finding)

Mechanism and evidence diverge early, converge deep



Layer 1 · r = 0.00

Native and Grad-CAM are decorrelated — the model looks broadly, but class evidence is tiny and sharp.

Layer 2 · undefined

Grad-CAM collapsed to an all-zero map for this image (ReLU'd gradients vanished) — a real, surfaced failure.

Layer 3 · r = 0.41

Mechanism starts aligning with evidence as receptive fields widen.

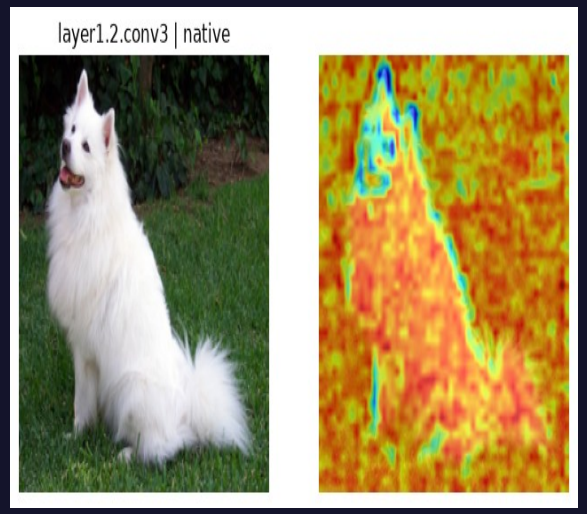
Layer 4 · r = 0.48

Deep features: where the model looks is where the decision lives.

Demo — M1 gate on Arjun



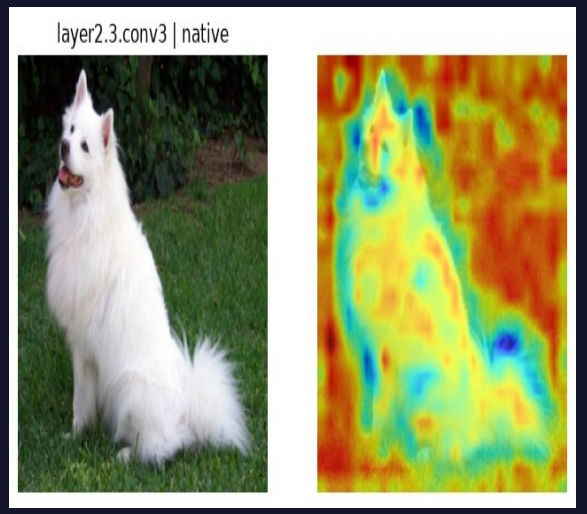
B1 · Grad-CAM



B1 · native



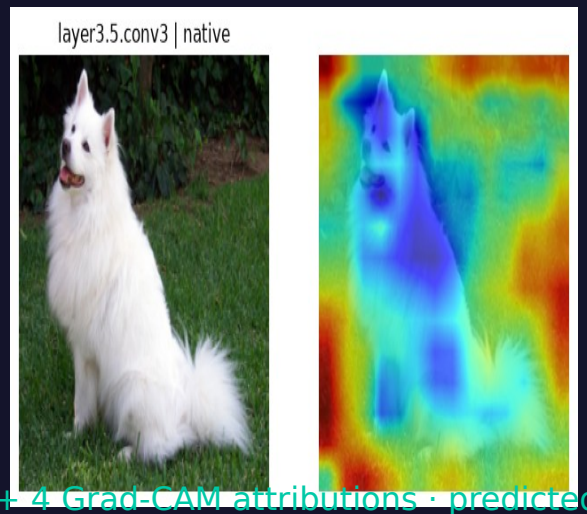
B2 · Grad-CAM



B2 · native



B3 · Grad-CAM



B3 · native



B4 · Grad-CAM



B4 · native

One forward pass + 4 Grad-CAM attributions · predicted Samoyed (correct) · CPU, torch 2.13 · commit 96c50b6

Results & Next Steps

M1 delivered; M2 opens the ViT frontier

Delivered (M1)

- Unified LayerMap / ModelInspection schema — tested (7 green)
- ResNet-50 dual-view extractor — native + Grad-CAM, 4 blocks
- Preprocessing with exact round-trip visualization
- End-to-end demo: Samoyed, 8 maps, correct prediction
- Hermetic test suite — runs without downloads

Roadmap

- M2: ViT extractor — attention maps through the same LayerMap
- M3: attention-quality metrics (entropy, locality, faithfulness)
- M4: screening pipeline + planted-bias & ObjectNet validation
- M5: interactive UI (Gradio) + intervention experiments

Honest open items: zero-map degeneracy at layer 2 needs a stability guard; GPU/device portability is code-ready, not yet exercised at scale.

Thank you

Questions & discussion

